

Laboratory Work #2

Part 1: Normalization

Task 1: Identify Normal Form Violation

- Violation: The table violates 1NF (First Normal Form).
- Explanation: The column seat_numbers stores non-atomic, comma-separated values (e.g., "12A, 12B") instead of a single value per row. Because it violates 1NF, it is also not in 2NF or 3NF.

Task 2: Step-by-Step Normalization (Tables)

Step 1: First Normal Form (1NF)

Eliminate non-atomic values by giving each seat its own row.

Booking_receipt_1NF

Primary Key: (booking_id, seat_number)

booking_id (PK)	seat_number (PK)	passenger_full_name	passenger_passport_number	flight_number	departure_airport_name	departure_city	arrival_airport_name	arrival_city	airline_name	ticket_price
1001	12A	Azat Erdos	N1234567	KC101	Almaty Int	Almaty	Astana Int	Astana	Air Astana	120.00
1001	12B	Aman Doszhan	N1234566	KC101	Almaty Int	Almaty	Astana Int	Astana	Air Astana	120.00

Step 2: Second Normal Form (2NF)

Remove partial functional dependencies so attributes depending only on booking_id are in a separate table.

Table 1: Booking_Seat

Primary Key: (booking_id, seat_number)

Foreign Key: booking_id [FK] → Booking_Details(booking_id)

booking_id (PK, FK)	seat_number (PK)
1001	12A
1001	12B

Table 2: Booking_Details
Primary Key: booking_id

booking_id (PK)	passenger_full_name	passenger_passport_number	flight_number	departure_airport_name	departure_city	arrival_airport_name	arrival_city	airline_name	ticket_price
1001	Azat Erdos	N1234567	KC101	Almaty Inter.	Almaty	Astana Inter.	Astana	Air Astana	120.00

Step 3: Third Normal Form (3NF)

Remove transitive dependencies by separating passengers, airports, flights, and bookings into their own tables.

Table 1: Passenger

Primary Key: passenger_passport_number

passenger_passport_number (PK)	passenger_full_name
N1234567	Azat Erdos

Table 2: Airport

Primary Key: airport_name

airport_name (PK)	city
Almaty Inter.	Almaty
Astana Inter.	Astana

Table 3: Flight

Primary Key: flight_number

Foreign Keys: departure_airport_name [FK] → Airport, arrival_airport_name [FK] → Airport

flight_number (PK)	airline_name	departure_airport_name [FK]	arrival_airport_name [FK]
KC101	Air Astana	Almaty Int	Astana Int

Table 4: Booking

Primary Key: booking_id

Foreign Keys: passenger_passport_number [FK] → Passenger, flight_number [FK] → Flight

booking_id (PK)	passenger_passport_number [FK]	flight_number [FK]	ticket_price
1001	N1234567	KC101	120.00

Table 5: Booking_Seat

Primary Key: (booking_id, seat_number)

Foreign Key: booking_id [FK] → Booking(booking_id)

booking_id (PK, FK)	seat_number (PK)
1001	12A
1001	12B

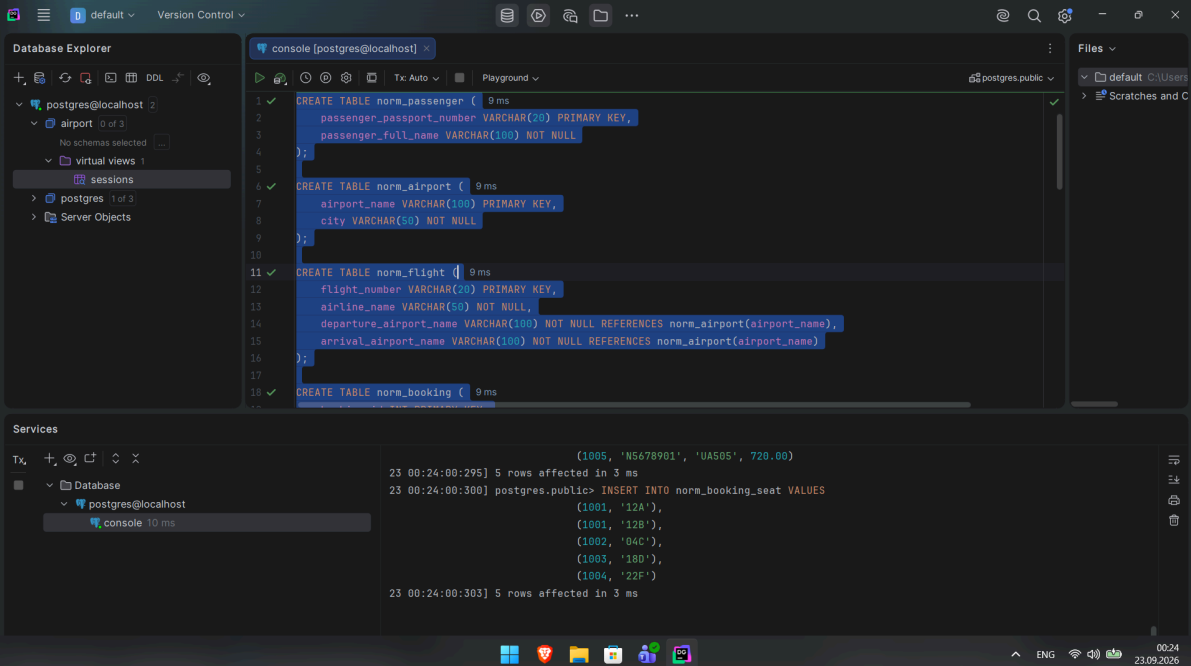
Task 3: Functional Dependencies

- {booking_id, seat_number} → ticket_price
 - booking_id → passenger_passport_number, flight_number, ticket_price
 - passenger_passport_number → passenger_full_name
 - flight_number → airline_name, departure_airport_name, arrival_airport_name
 - departure_airport_name → departure_city
 - arrival_airport_name → arrival_city
-

Task 4: 3NF Check for Flights Table

- Table: Flights(flight_id, sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, act_departure_time, act_arrival_time, created_at, updated_at)
- Primary Key: flight_id
- Functional Dependencies: flight_id determines all other columns in the table.
- Transitive Dependency Check: No transitive dependency exists between non-key attributes. Attributes like airport name/city or airline details are not stored in Flights; they are referenced via foreign key IDs (departing_airport_id, arriving_airport_id, airline_id) pointing to Airport and Airline. Therefore, Flights is in 3NF.

Task 5: 3NF Implementation & Sample Data



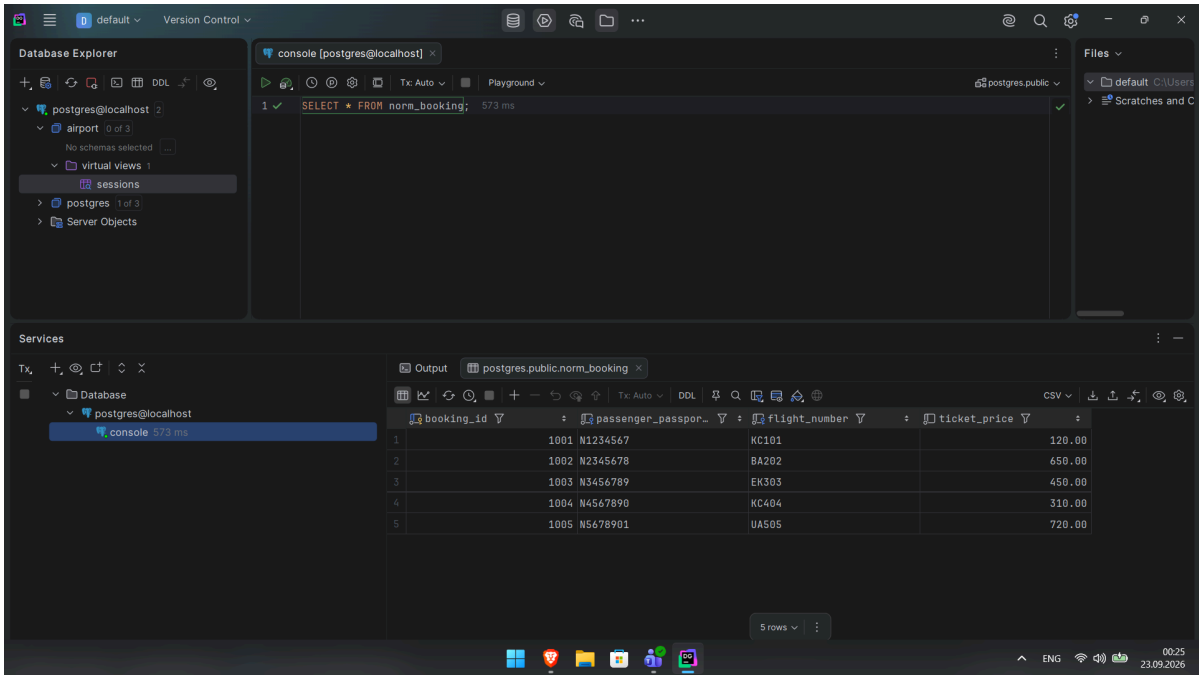
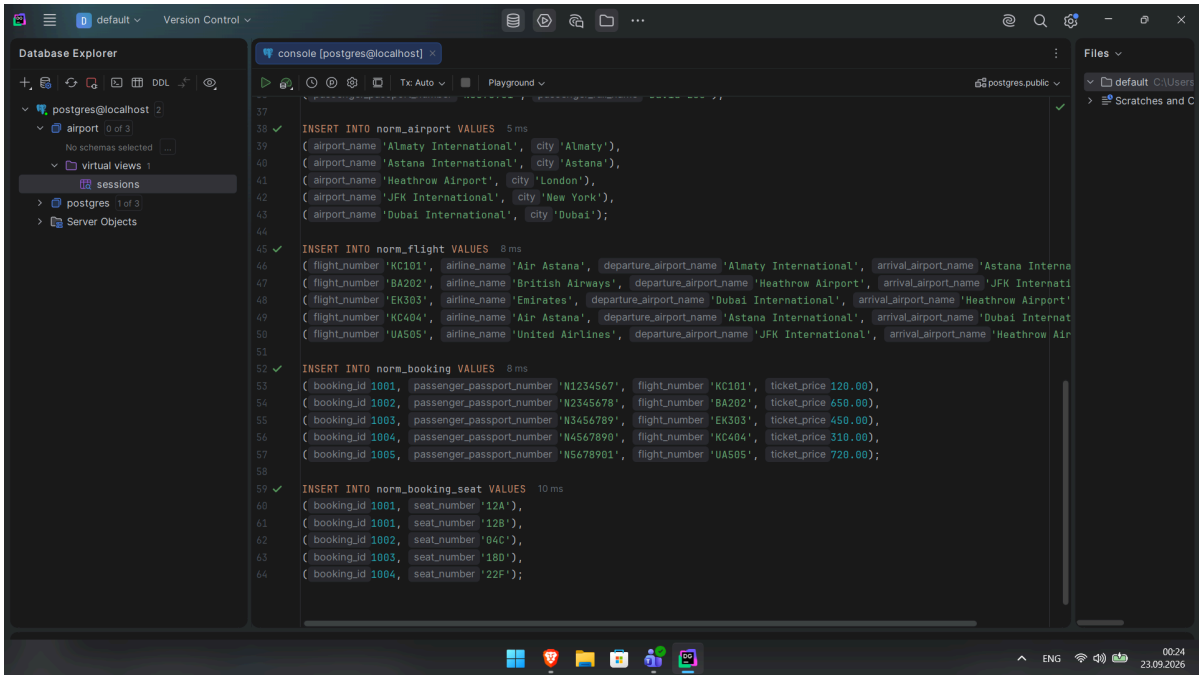
The screenshot displays a database management interface with a dark theme. On the left, the 'Database Explorer' shows a connection to 'postgres@localhost' with a tree view containing 'airport', 'virtual views', 'sessions', and 'Server Objects'. The main console window shows the following SQL queries and their execution results:

```
1 CREATE TABLE norm_passenger ( 9 ms
2   passenger_passport_number VARCHAR(20) PRIMARY KEY,
3   passenger_full_name VARCHAR(100) NOT NULL
4 );
5
6 CREATE TABLE norm_airport ( 9 ms
7   airport_name VARCHAR(100) PRIMARY KEY,
8   city VARCHAR(50) NOT NULL
9 );
10
11 CREATE TABLE norm_flight ( 9 ms
12   flight_number VARCHAR(20) PRIMARY KEY,
13   airline_name VARCHAR(50) NOT NULL,
14   departure_airport_name VARCHAR(100) NOT NULL REFERENCES norm_airport(airport_name),
15   arrival_airport_name VARCHAR(100) NOT NULL REFERENCES norm_airport(airport_name)
16 );
17
18 CREATE TABLE norm_booking ( 9 ms
```

The execution results for the last query are shown below:

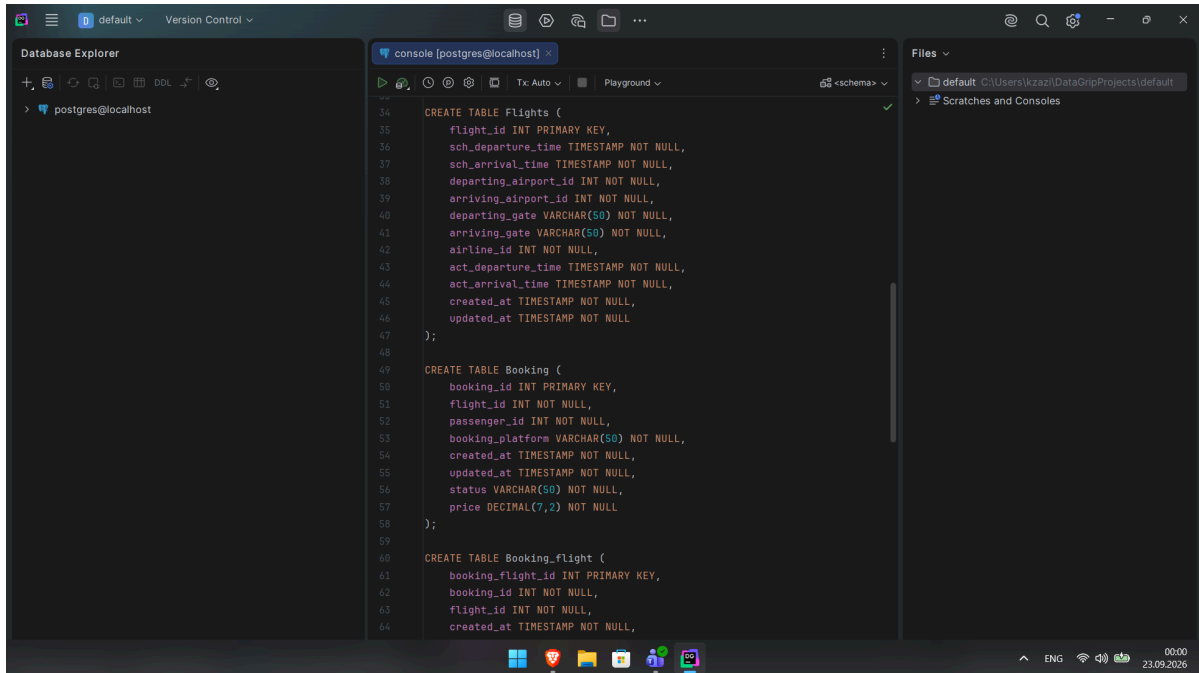
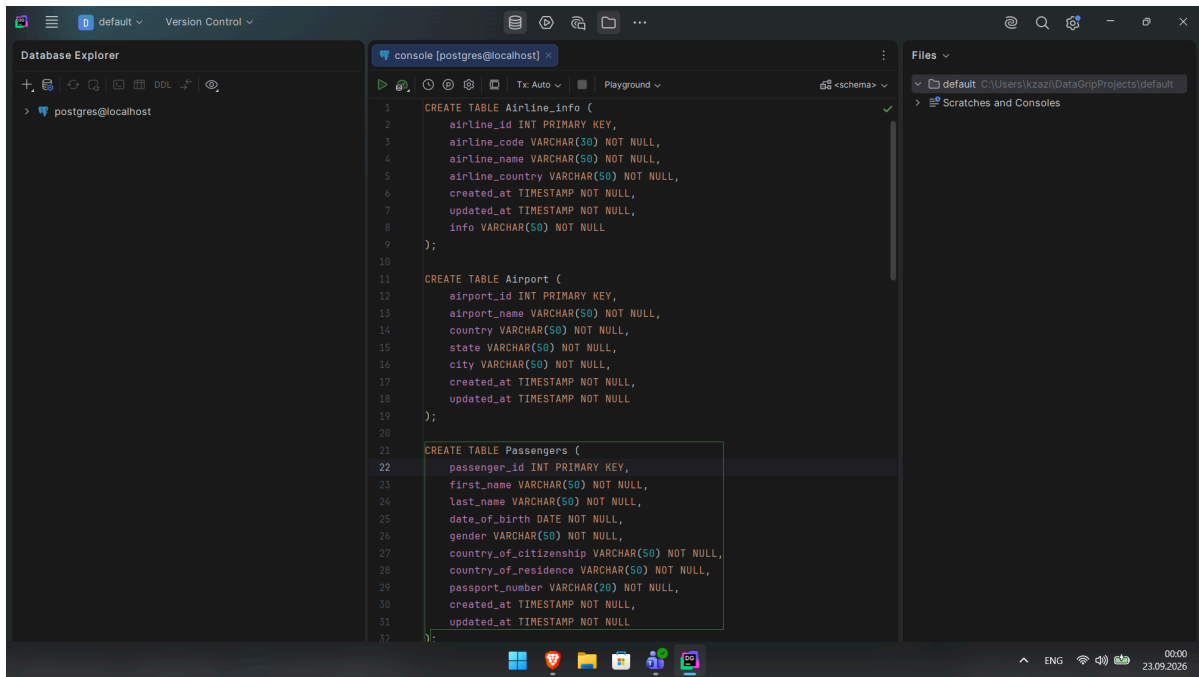
```
(1005, 'N5678901', 'UAS05', 720.00)
23 00:24:00:295] 5 rows affected in 3 ms
23 00:24:00:300] postgres.public> INSERT INTO norm_booking_seat VALUES
(1001, '12A'),
(1001, '12B'),
(1002, '04C'),
(1003, '18D'),
(1004, '22F')
23 00:24:00:303] 5 rows affected in 3 ms
```

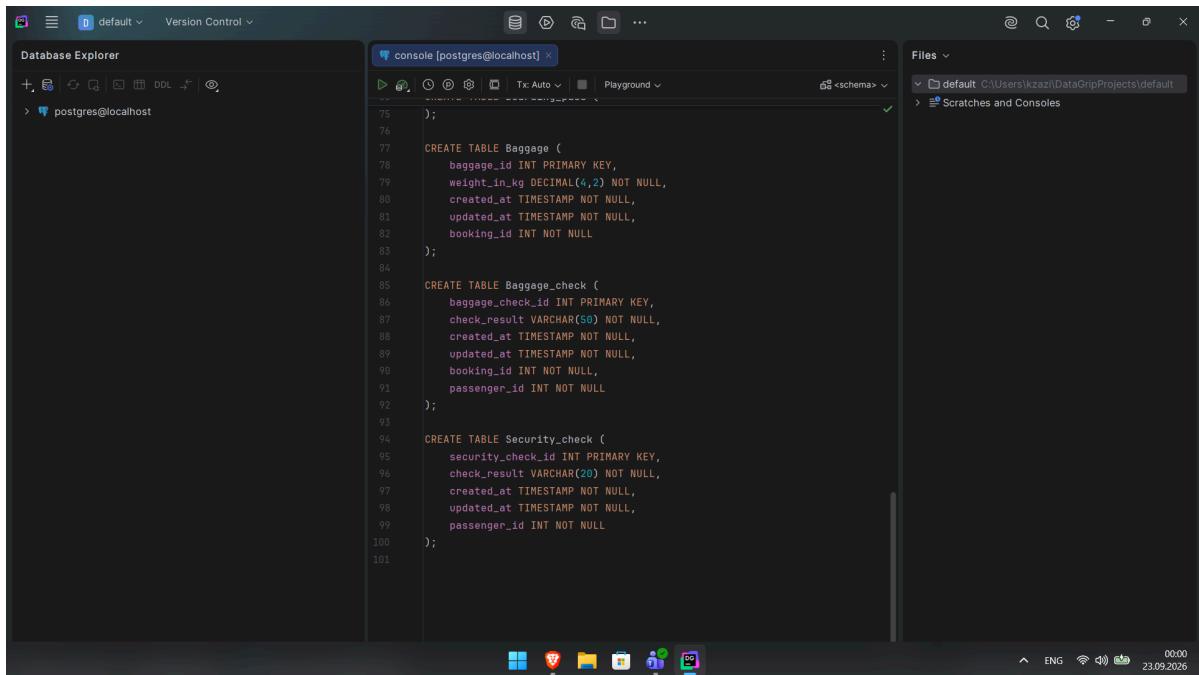
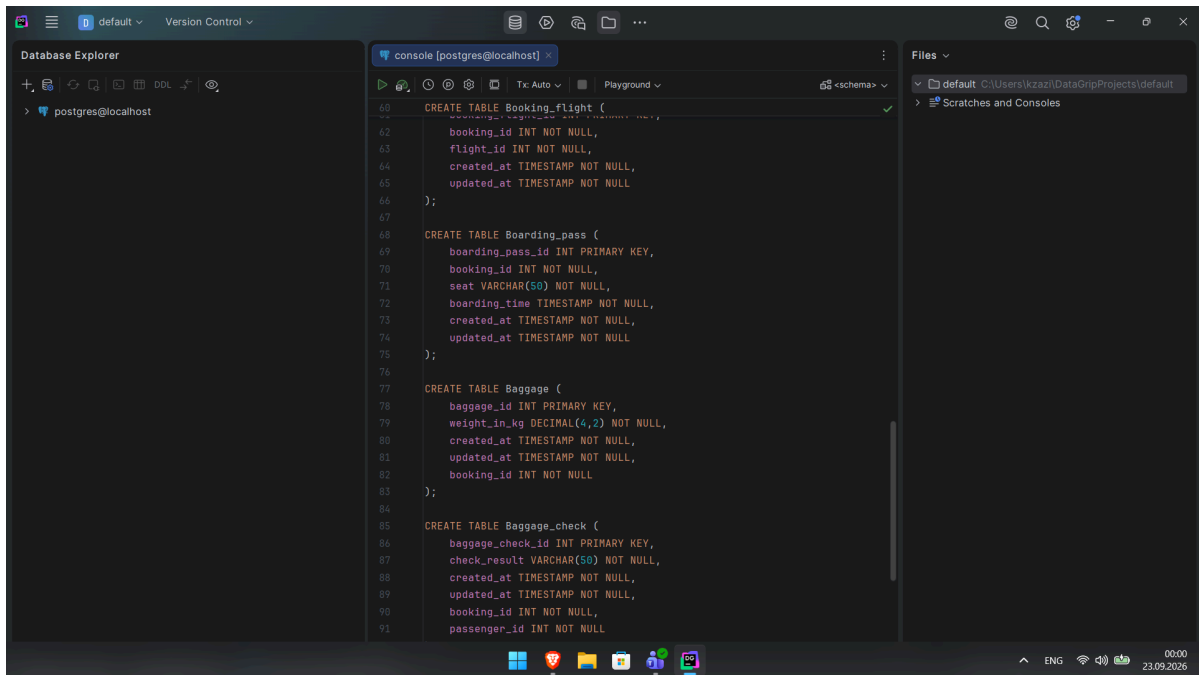
The bottom status bar shows the system time as 00:24 on 23.09.2026.



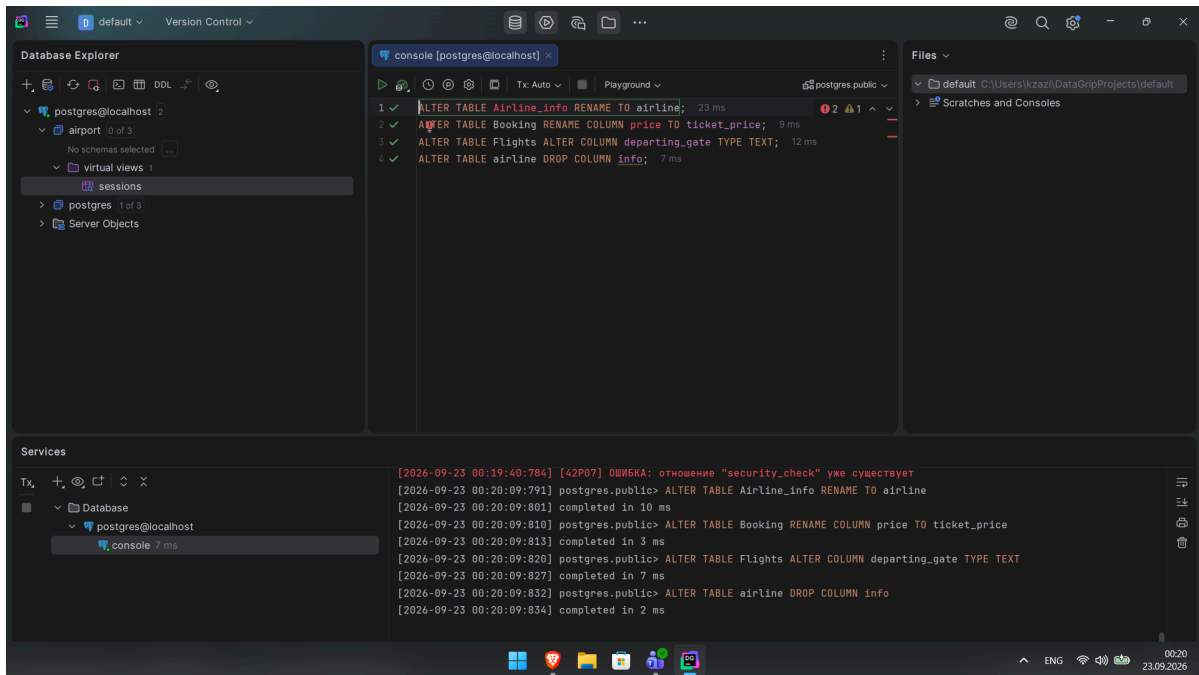
Part 2: DDL

1. Create Tables





2. Alter Tables



3. Add Foreign Keys

